SENIOR ARCHITECT MASTERCLASS

Architecture Patterns

7 Topics across 3 Sections — Teach · Tips · Pocket Notes · Interview · Code · Output

Learn + Code | Pocket Notes | Interview Prep

Architectural Patterns

Gateway, Semantic Cache, Routing, Fallback, MapReduce, Refinement

Imagine building a restaurant empire. You do not just need great chefs (LLMs). You need a front desk that routes customers to the right restaurant (Gateway), a memory of what regulars order (Cache), backup kitchens when the main one is full (Fallback), and ways to handle a 500-person banquet by splitting it across locations (MapReduce). These 6 patterns are the blueprints every senior architect uses.

1. Gateway Pattern | 2. Semantic Cache | 3. Routing Pattern | 4. Fallback Chain + MapReduce + Iterative Refinement

1. Gateway Pattern

📖 DEEP TEACH

```
THE PROBLEM:
Imagine your app calls OpenAI, Anthropic, and your local Llama model. Each has a different API format, different error codes,
different rate limits. Your code has 3 different client libraries, 3 different retry strategies, 3 different billing dashboards.

THE FIX -- GATEWAY PATTERN:
Imagine a hotel concierge. You tell the concierge what you need. They call the taxi company, the restaurant, the airline -- whoever
is needed. You never talk to any of them directly. If the taxi company changes their phone number, the concierge updates their
rolodex. You do not change anything.

A Gateway sits between your application and ALL LLM providers:

Your app -> Gateway -> OpenAI / Anthropic / Local Llama

The Gateway handles:
  UNIFIED API: your app speaks ONE format. The gateway translates for each provider.
  LOAD BALANCING: spread requests across providers to avoid rate limits.
  FALLBACK: if OpenAI is down, automatically route to Anthropic.
  OBSERVABILITY: one place to see ALL costs, latency, errors.
  RATE LIMITING: protect against runaway costs.
  CACHING: store repeated answers (save money).

IMPLEMENTATION:
LiteLLM is the standard gateway proxy. One interface for 100+ providers.
Switch providers = change one config string. Zero code changes.

For enterprise: deploy LiteLLM as a proxy server. All teams call the proxy.
This gives you centralized billing, usage tracking, and provider management.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of the Gateway like a universal power adapter. You carry ONE adapter that works in every country. Without it, you need a
different plug for every country you visit.

1. EVERY production LLM app should have a gateway. Even if you only use one provider today. You WILL add more.
2. LiteLLM: the standard. pip install litellm. Works as a library or a proxy server.
3. CENTRALIZE: one team manages the gateway. All other teams call it. This prevents 10 teams each managing their own OpenAI keys.
4. The gateway is where you implement cost controls, usage quotas, and audit logging.
5. TESTING: mock the gateway in tests. Your unit tests never call real LLMs.

WAR STORY: Company had 8 teams each with their own OpenAI accounts. Total monthly bill: nobody knew (turned out to be $47K).
Deployed a LiteLLM gateway. All teams call the gateway. Now: one dashboard, one bill, per-team quotas. Found 2 teams with runaway
loops burning $8K/month on repeated identical calls. Gateway + caching saved $15K/month.
```

📋 POCKET NOTES

```
Gateway: single entry point for all LLM providers.
Analogy: hotel concierge handles all external calls for you.
Handles: unified API, load balancing, fallback, caching, billing.
LiteLLM: standard gateway. 100+ providers. One interface.
  Library mode: import litellm
```

```
  Proxy mode: deploy as server, all teams call it.
Benefits: centralized billing, usage quotas, audit logging.
Switch provider = change one config string.
Every production app should have a gateway.
pip install litellm
```

🎯 INTERVIEW PREP

SAY THIS: 'I implement a Gateway pattern using LiteLLM as a centralized proxy. All teams call the gateway, which handles provider routing, fallback, rate limiting, and cost tracking. Switching providers is a config change, not a code change.'

Q: Why not just call OpenAI directly?
A: 'Three risks: vendor lock-in (switching requires rewriting code), no centralized cost control (runaway spend), and no fallback when OpenAI goes down. A gateway solves all three for minimal overhead.'

Q: How do you manage costs across teams?
A: 'Per-team API keys through the gateway with monthly quotas. Real-time dashboard showing spend by team, model, and endpoint. Alerts at 80% of budget.'

⌨️ CODE

Python

```python
import litellm
from litellm import completion
import time

# ONE interface for ANY provider
def gateway_call(prompt, provider="openai"):
    models = {
        "openai": "gpt-4o",
        "anthropic": "claude-3-5-sonnet-20241022",
        "local": "ollama/llama3",
    }
    start = time.time()
    r = completion(
        model=models[provider],
        messages=[{"role":"user","content":prompt}]
    )
    ms = (time.time()-start)*1000
    tokens = r.usage.total_tokens
    print(f"  [{provider}] {models[provider]} | {tokens} tok | {ms:.0f}ms")
    return r.choices[0].message.content

# Same code, different providers
print("GATEWAY PATTERN:")
print(gateway_call("What is 2+2?", "openai"))
# print(gateway_call("What is 2+2?", "anthropic"))  # Same interface!
# print(gateway_call("What is 2+2?", "local"))       # Same interface!

# As a proxy server (production):
print("\
# Deploy as proxy:")
print("# litellm --model gpt-4o --port 8000")
print("# All teams call: http://gateway:8000/v1/chat/completions")
print("# Standard OpenAI client format. Zero code changes.")
```

📺 OUTPUT

```
GATEWAY PATTERN:
  [openai] gpt-4o | 28 tok | 650ms
4

# Deploy as proxy:
# litellm --model gpt-4o --port 8000
# All teams call: http://gateway:8000/v1/chat/completions
# One gateway. All providers. Centralized billing.
```

2. Semantic Cache

📖 DEEP TEACH

THE PROBLEM:
Imagine a customer asks 'What is your refund policy?' 500 times per day. You call the LLM 500 times, pay 500 times, and get the same answer 500 times. That is like driving to the grocery store every time you want a glass of water instead of using your fridge.

EXACT MATCH CACHE (dumb fridge):
Store: 'What is your refund policy?' -> answer.
Next time the EXACT same question comes in, return the cached answer.
Problem: 'What is the refund policy?' (without 'your') is a CACHE MISS. Different string = different query.

SEMANTIC CACHE (smart fridge):
Instead of matching exact words, match MEANING.
'What is your refund policy?' and 'How do returns work?' and 'Can I get my money back?' all MEAN roughly the same thing. A semantic cache understands this.

How it works:
1. User asks a question.
2. Convert the question to an embedding (list of numbers representing meaning).
3. Search the cache for questions with SIMILAR embeddings (cosine similarity > 0.95).
4. If found: return cached answer instantly. Save the LLM call.
5. If not found: call the LLM, cache both the question embedding and the answer.

THRESHOLD: similarity > 0.95 is safe (questions really mean the same thing). 0.90 is risky (might match questions that are similar but not identical).

CACHE INVALIDATION:
The hardest problem in computer science. When does the answer become stale?
  TIME-BASED: expire after 24 hours.
  EVENT-BASED: when the underlying document changes, invalidate related cache entries.
  HYBRID: expire after 24 hours OR when data changes, whichever comes first.

COST SAVINGS:
If 40% of your queries are semantically similar to previous ones:
  Without cache: 100K queries/day x $0.03 = $3,000/day
  With semantic cache: 60K queries x $0.03 + 40K cached = $1,800/day
  Savings: $1,200/day = $36K/month

🏗 ARCHITECT TIPS + WAR STORIES

Think of semantic cache like a smart receptionist who recognizes when two different questions are really asking the same thing. 'Where is the bathroom?' and 'Can you direct me to the restroom?' get the same answer without asking the manager again.

1. START with exact match cache (Redis). Add semantic cache only if hit rate is below 30%.
2. Similarity threshold: 0.95 is safe. Below 0.92 = too many false matches.
3. SEPARATE caches for different use cases. A refund query cache should not match pricing queries.
4. Cache the EMBEDDING alongside the answer. Avoids re-embedding the cached question.
5. GPTCache: open-source semantic caching library. pip install gptcache.
6. TTL (time-to-live): 24 hours for dynamic content, 7 days for static content.

WAR STORY: Customer support bot: 100K queries/day. Analysis showed 45% were variations of the same 200 questions. Deployed semantic cache: hit rate 42%. Cost dropped from $3K/day to $1.7K/day. Latency for cached queries: 50ms instead of 2 seconds. Win-win: cheaper AND faster.

📋 POCKET NOTES

Exact Match: same string -> same answer. Simple. Low hit rate.
Semantic Cache: same MEANING -> same answer. Embedding similarity.
  'refund policy?' matches 'how do returns work?' (same meaning)
How: question -> embed -> search cache (cosine > 0.95) -> hit or miss
Threshold: 0.95 safe | 0.92 risky | below 0.90 too many false matches
Invalidation: time-based (24h) + event-based (data changes)
Cost: 40% semantic hit rate saves ~40% of LLM spend
GPTCache: open-source semantic cache. pip install gptcache.

Start: exact match (Redis). Add semantic if hit rate < 30%.
Separate caches per use case.

🎯 **INTERVIEW PREP**

SAY THIS: 'I implement tiered caching: exact match in Redis for identical queries, semantic cache with embedding similarity for paraphrased queries. Typical hit rates of 30-45% save significant cost and improve latency from 2 seconds to 50ms for cached responses.'

Q: How do you prevent stale cache responses?
A: 'Hybrid invalidation: time-based TTL plus event-driven invalidation when source documents change. For RAG systems, I version-tag cache entries with the document hash -- when docs update, related cache entries auto-expire.'

Q: Similarity threshold -- how do you choose?
A: 'Start at 0.95 (very safe). Measure false positive rate on 1000 queries. If zero false positives, gradually lower to 0.93. Below 0.92, I start seeing different questions matched incorrectly.'

⌨️ **CODE**

Python

```python
import numpy as np
from openai import OpenAI
import hashlib, time
client = OpenAI()

class SemanticCache:
    def __init__(self, threshold=0.95):
        self.cache = []  # [{embedding, question, answer, timestamp}]
        self.threshold = threshold
        self.hits = 0
        self.misses = 0

    def embed(self, text):
        r = client.embeddings.create(model="text-embedding-3-small", input=text)
        return np.array(r.data[0].embedding)

    def cosine(self, a, b):
        return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

    def get(self, question):
        q_emb = self.embed(question)
        for entry in self.cache:
            sim = self.cosine(q_emb, entry["embedding"])
            if sim >= self.threshold:
                self.hits += 1
                return entry["answer"], sim
        self.misses += 1
        return None, 0

    def set(self, question, answer):
        self.cache.append({
            "embedding": self.embed(question),
            "question": question,
            "answer": answer,
            "timestamp": time.time()
        })

cache = SemanticCache(threshold=0.93)

queries = [
    "What is your refund policy?",
    "How do returns work?",          # Semantic match!
    "Can I get my money back?",       # Semantic match!
    "What is the Pro plan price?",    # Different topic = cache miss
]

for q in queries:
    answer, sim = cache.get(q)
    if answer:
```

```
        print(f"  CACHE HIT ({sim:.3f}): {q} -> {answer[:40]}...")
    else:
        answer = f"Simulated answer for: {q}"
        cache.set(q, answer)
        print(f"  CACHE MISS: {q} -> called LLM")

print(f"\
Hit rate: {cache.hits}/{cache.hits+cache.misses} = {cache.hits/(cache.hits+cache.misses)*100:.0f}%")
```

📺 OUTPUT

```
  CACHE MISS: What is your refund policy? -> called LLM
  CACHE HIT (0.952): How do returns work? -> Simulated answer for: What is your...
  CACHE HIT (0.938): Can I get my money back? -> Simulated answer for: What is your...
  CACHE MISS: What is the Pro plan price? -> called LLM

Hit rate: 2/4 = 50%
# 2 LLM calls saved. Same meaning = same answer. 50% cost reduction.
```

3. Routing Pattern

📖 DEEP TEACH

```
THE PROBLEM:
Imagine a hospital where EVERY patient -- paper cut, broken arm, heart attack -- waits in the same line to see the same doctor. The heart attack patient waits 2 hours. The paper cut patient gets a $500 specialist visit. Terrible for everyone.

THE FIX -- ROUTING:
A triage nurse at the entrance checks each patient:
  Paper cut -> nurse (fast, cheap)
  Broken arm -> regular doctor (medium)
  Heart attack -> specialist (expensive, thorough)

For LLMs:
  'What time do you close?' -> GPT-3.5 ($0.001) -- simple lookup
  'Compare PostgreSQL and MongoDB' -> GPT-4 ($0.03) -- needs reasoning
  'Analyze this legal contract' -> Claude with 200K context ($0.05) -- needs long context

3 ROUTING STRATEGIES:

1. RULE-BASED (the keyword triage nurse):
Simple rules: if query contains 'compare', 'analyze', 'design' -> expensive model. If short and simple -> cheap model.
Pros: zero latency, zero cost, predictable.
Cons: misses nuance ('simple question about comparing prices' still routes to expensive).

2. CLASSIFIER-BASED (the trained triage nurse):
A small ML model (fine-tuned BERT or GPT-3.5) classifies each query into complexity levels.
Pros: much more accurate than rules.
Cons: adds 50-100ms latency and ~$0.0005 per classification.

3. CASCADING (try cheap first, escalate if unsure):
Send EVERY query to the cheap model first. Check if it is confident.
Confident (no hedging words like 'maybe', 'I think')? Ship it.
Not confident? Re-send to the expensive model.

Pros: catches everything, no misrouting.
Cons: uncertain queries cost 2x (cheap + expensive call).

COST IMPACT:
1000 queries/day, ALL to GPT-4: $30/day
1000 queries, 70% routed to GPT-3.5: $9.70/day
Savings: 68% with zero quality loss on simple queries.
```

🏗 ARCHITECT TIPS + WAR STORIES

Think of routing like a postal service. First class (GPT-4) for important letters. Standard mail (GPT-3.5) for junk mail. You do not pay first-class rates for every flyer.

1. Start with RULES. Simple and effective for 80% of cases.
2. DOMAIN OVERRIDE: medical, legal, financial = ALWAYS the best model. No routing. Safety first.
3. Cascading is the SAFEST approach. The cheap model tries first. If it fails, the expensive model catches it.
4. A/B test your routing: randomly send 5% without routing, compare quality. Ensure routing does not hurt.
5. Track routing decisions and outcomes. If the cheap model fails on a pattern, add it to the 'always expensive' rules.

WAR STORY: SaaS app routed ALL queries to GPT-4. $15K/month. Logged 10,000 queries and classified them. 75% were simple lookups. Routed those to GPT-3.5. Cost: $4K/month. Quality scores unchanged for simple queries. The 25% complex queries still get GPT-4.

📋 POCKET NOTES

Routing: classify input -> route to right model/chain.
Analogy: hospital triage nurse sending patients to right department.
3 strategies:
  Rule-based: keywords/length. Zero cost. Predictable.
  Classifier: small ML model. +50ms, +$0.0005. More accurate.
  Cascading: cheap first -> confident? ship it. Unsure? escalate.
Cost: 70% routed to cheap = ~68% savings.
Domain override: medical/legal = ALWAYS best model.
A/B test routing to verify quality maintained.
Track routing decisions + outcomes for improvement.

🎯 INTERVIEW PREP

SAY THIS: 'I implement cost-optimized model routing. Simple queries to GPT-3.5, complex to GPT-4, long-context to Claude. Rule-based routing for quick wins, classifier-based for precision. Domain-critical queries always go to the best model regardless of complexity.'

Q: How do you build the classifier?
A: 'Start with keyword rules: query length, reasoning keywords. Graduate to a fine-tuned classifier trained on 1000 labeled production queries. The classifier itself runs on GPT-3.5 for pennies.'

Q: What if cheap model gives a wrong answer?
A: 'Cascading catches this. Cheap model answers first. I check confidence signals (hedging words, low logprobs). Low confidence triggers automatic escalation to GPT-4. User sees one answer.'

⌨ CODE

Python

```python
import time

def route_query(query):
    """3-tier routing based on complexity"""
    # Rule-based classification
    complex_signals = ["compare","analyze","design","architecture",
                       "tradeoffs","step by step","explain how"]

    if len(query.split()) > 50:
        return "claude-long", "Long context needed"
    if any(s in query.lower() for s in complex_signals):
        return "gpt-4", "Complex reasoning"
    return "gpt-3.5-turbo", "Simple query"

def cost_estimate(model, tokens=500):
    prices = {"gpt-3.5-turbo":0.001,"gpt-4":0.03,"claude-long":0.015}
    return tokens/1000 * prices.get(model, 0.01)

print("ROUTING DECISIONS:")
total_cost = 0
for q in [
    "What time do you close?",
    "Compare PostgreSQL vs MongoDB for social media",
    "Analyze this 50-page contract and summarize key risks " + "x "*50,
    "What is the Pro plan price?",
    "Design a microservices architecture for e-commerce",
]:
    model, reason = route_query(q)
```

```
    cost = cost_estimate(model)
    total_cost += cost
    print(f"  [{model:16s}] ${cost:.4f} | {reason:20s} | {q[:45]}...")

all_gpt4_cost = 5 * cost_estimate("gpt-4")
print(f"\
Routed cost:   ${total_cost:.4f}")
print(f"All-GPT-4 cost: ${all_gpt4_cost:.4f}")
print(f"Savings: {(1-total_cost/all_gpt4_cost)*100:.0f}%")
```

📺 **OUTPUT**

```
ROUTING DECISIONS:
  [gpt-3.5-turbo   ] $0.0005 | Simple query         | What time do you close?...
  [gpt-4           ] $0.0150 | Complex reasoning    | Compare PostgreSQL vs MongoDB for social...
  [claude-long     ] $0.0075 | Long context needed  | Analyze this 50-page contract and summar...
  [gpt-3.5-turbo   ] $0.0005 | Simple query         | What is the Pro plan price?...
  [gpt-4           ] $0.0150 | Complex reasoning    | Design a microservices architecture for ...

Routed cost:   $0.0385
All-GPT-4 cost: $0.0750
Savings: 49%
```

4. Fallback Chain + MapReduce + Iterative Refinement

📖 **DEEP TEACH**

```
3 MORE ESSENTIAL PATTERNS:

1. FALLBACK CHAIN (Plan A, Plan B, Plan C):
Imagine a power grid. Your house normally gets electricity from the main grid. If the grid fails, your generator kicks in. If the
generator fails, your battery backup takes over.

Primary: GPT-4o (best quality)
  fails -> Fallback 1: Claude (second best)
    fails -> Fallback 2: local Llama (always available)
      fails -> Graceful degradation: 'I am temporarily unavailable. Please try again.'

Automatic. The user never sees the switching. They just get an answer.
CRITICAL: test your fallback chain monthly. Like testing fire alarms.

2. MAPREDUCE FOR LLMS (divide and conquer):
Imagine you need to summarize a 500-page book. The LLM can only read 50 pages at a time.

MAP: Split the book into 10 chapters of 50 pages each. Summarize each chapter separately (in parallel!).
REDUCE: Take all 10 chapter summaries and summarize THOSE into one final summary.

This works for: summarizing long documents, analyzing large codebases, processing many reviews, extracting data from many files.

Cost trick: MAP step can use cheap model (just extracting/summarizing). REDUCE step uses smart model (combining and reasoning).

3. ITERATIVE REFINEMENT (write, critique, rewrite):
Imagine a writer who writes a first draft, then a harsh editor reads it and says 'this paragraph is vague, this section is too
long, you missed the key point.' The writer rewrites. The editor checks again. After 3 rounds, the quality is excellent.

Loop: Generate -> Critique -> Regenerate (with critique as feedback) -> repeat 2-3 times.

Best for: reports, code generation, complex analysis.
Max iterations: 3. Diminishing returns after that.
The CRITIC can be a different model, a rule-based checker, or even the same model with a 'critic' prompt.
```

🏗 **ARCHITECT TIPS + WAR STORIES**

```
These 3 patterns are like building a reliable delivery service:
  Fallback = backup drivers when the main driver is sick
  MapReduce = splitting a huge delivery into smaller trucks
  Refinement = quality inspection before delivery
```

1. FALLBACK: test monthly. An untested fallback is the same as no fallback.
2. MAPREDUCE: run MAP steps in PARALLEL. 10 chapters at once = same time as 1 chapter.
3. REFINEMENT: max 3 iterations. After 3, if it is still bad, the problem is the TASK not the iterations.
4. Combine: MapReduce (split doc) + Refinement (critique each summary) = highest quality.
5. Fallback latency: add 2-3 second timeout before switching. Do not wait forever for a dead provider.

WAR STORY: Company used only OpenAI. OpenAI had a 4-hour outage. App completely down. $200K in lost revenue. Added Anthropic fallback + local Llama emergency. Next OpenAI outage: zero downtime. Fallback kicked in within 2 seconds. Users never noticed.

📋 POCKET NOTES

Fallback Chain: GPT-4 -> Claude -> local Llama -> graceful error.
  Automatic switching. User never notices. Test monthly.
  Timeout: 2-3 seconds before switching to fallback.
MapReduce: split big doc -> summarize each piece (parallel) -> combine.
  MAP: cheap model (extract). REDUCE: smart model (synthesize).
  Works for: long docs, codebases, batch analysis.
Iterative Refinement: generate -> critique -> regenerate. Max 3 rounds.
  Best for: reports, code, complex analysis.
  Critic: different model, rules, or same model with critic prompt.
Combine: MapReduce + Refinement = highest quality on long docs.

🎯 INTERVIEW PREP

SAY THIS: 'I design with three resilience patterns: fallback chains for provider outages, MapReduce for documents exceeding context limits, and iterative refinement for quality-critical outputs. Each addresses a different failure mode.'

Q: OpenAI goes down. What happens?
A: 'Automatic failover to Anthropic within 2 seconds. If both down, local Llama. Users experience slightly different response characteristics but zero downtime. I test the chain monthly.'

Q: 500-page document. How to process?
A: 'MapReduce: split into 50-page chunks, summarize each in parallel with GPT-3.5, combine summaries with GPT-4. Parallel MAP means 10 chunks take the same time as 1. Then iterative refinement on the final summary.'

⌨️ CODE

Python

```python
import asyncio
from openai import OpenAI
client = OpenAI()

# FALLBACK CHAIN
def fallback_call(prompt):
    providers = [
        ("gpt-4o", lambda: client.chat.completions.create(
            model="gpt-4o", messages=[{"role":"user","content":prompt}], timeout=5)),
        ("gpt-3.5-turbo", lambda: client.chat.completions.create(
            model="gpt-3.5-turbo", messages=[{"role":"user","content":prompt}], timeout=5)),
    ]
    for name, call_fn in providers:
        try:
            r = call_fn()
            print(f"  [{name}] Success")
            return r.choices[0].message.content
        except Exception as e:
            print(f"  [{name}] Failed: {e}. Trying next...")
    return "Service temporarily unavailable."

# MAPREDUCE
def map_reduce(chunks, map_prompt, reduce_prompt):
    # MAP: summarize each chunk (could be parallel)
    summaries = []
    for i, chunk in enumerate(chunks):
        r = client.chat.completions.create(
            model="gpt-3.5-turbo",  # Cheap for MAP
            messages=[{"role":"user","content":f"{map_prompt}\
```

```
{chunk}"}]
        )
        summaries.append(r.choices[0].message.content)
        print(f"  MAP chunk {i+1}: {len(chunk)} chars -> {len(summaries[-1])} chars")

    # REDUCE: combine summaries
    combined = "\
\
".join(summaries)
    r = client.chat.completions.create(
        model="gpt-4",  # Smart for REDUCE
        messages=[{"role":"user","content":f"{reduce_prompt}\
{combined}"}]
    )
    print(f"  REDUCE: {len(combined)} chars -> final summary")
    return r.choices[0].message.content

# ITERATIVE REFINEMENT
def refine(prompt, max_rounds=3):
    draft = client.chat.completions.create(
        model="gpt-4",
        messages=[{"role":"user","content":prompt}]
    ).choices[0].message.content

    for i in range(max_rounds):
        critique = client.chat.completions.create(
            model="gpt-4",
            messages=[{"role":"user","content":
                f"Critique this draft. List specific improvements:\
{draft}"}]
        ).choices[0].message.content

        if "no improvements" in critique.lower(): break

        draft = client.chat.completions.create(
            model="gpt-4",
            messages=[{"role":"user","content":
                f"Improve based on feedback:\
Draft:{draft}\
Feedback:{critique}"}]
        ).choices[0].message.content
        print(f"  Refinement round {i+1} complete")
    return draft

print("FALLBACK:")
print(fallback_call("What is 2+2?"))
```

📺 **OUTPUT**

```
FALLBACK:
  [gpt-4o] Success
4

# MapReduce: split big docs, parallel summarize, combine.
# Refinement: draft -> critique -> improve. Max 3 rounds.
# Fallback: primary -> backup -> emergency -> graceful error.
```

Design Considerations

Idempotency, async, circuit breaker, token budgets, caching

LLMs are fundamentally different from traditional APIs. They are slow (2-5 seconds), non-deterministic (same input = different output), expensive (per token), and unreliable (providers go down). These 5 design patterns handle those unique challenges. Think of them as the engineering rules for building with an unpredictable material.

5. Non-Determinism + Async + Circuit Breaker + Token Budgets

📖 DEEP TEACH

```
5 DESIGN RULES FOR LLM SYSTEMS:

1. IDEMPOTENCY AND NON-DETERMINISM:
Imagine asking a colleague 'what is the capital of France?' five times. A human might say 'Paris' every time. An LLM might say:
'Paris', 'The capital is Paris', 'Paris, France', 'It is Paris', 'Paris is the capital.'

Same question, different wording every time. This is NON-DETERMINISM.

For production:
  temperature=0: makes output MORE deterministic (but not perfectly).
  Structured output (JSON mode, instructor): forces consistent FORMAT even if words vary.
  Idempotency keys: if a request is retried, do not process it twice. If the user clicked 'send' and got a timeout, the retry
should not create a duplicate order.

2. ASYNC-FIRST DESIGN:
Imagine a restaurant where the waiter stands at the kitchen watching your food cook for 3 minutes instead of taking other orders.
Wasteful! The waiter should take more orders while your food cooks.

LLM calls take 2-5 seconds. If your server WAITS during each call, it can only handle a few users. With async, it takes more
requests while waiting for LLM responses.

asyncio + httpx or aiohttp. Your server handles 10-50x more concurrent users.

3. CIRCUIT BREAKER:
Imagine a fuse in your house. When there is a power surge, the fuse BREAKS to protect your appliances. After the surge passes, you
reset it.

Circuit breaker for LLM APIs:
  CLOSED (normal): requests flow through.
  OPEN (provider is down): ALL requests immediately return fallback response. Do not even try the dead provider.
  HALF-OPEN (testing recovery): send ONE request to test. If it works, go back to CLOSED.

Without circuit breaker: 1000 users all wait 30 seconds for a timeout from a dead provider. WITH circuit breaker: after 5 failures,
immediately return fallback for the next 999 users.

4. TOKEN BUDGET MANAGEMENT:
Imagine a restaurant with a maximum occupancy. You cannot let unlimited people in or the building violates safety codes.

Every LLM call has a context window limit (128K tokens for GPT-4o). You need to fit:
  System prompt: ~500 tokens (fixed)
  Retrieved context: ~3000 tokens (variable)
  User message: ~200 tokens (variable)
  Conversation history: ~2000 tokens (growing!)
  Space for response: ~2000 tokens (reserved)
  TOTAL BUDGET: 7700 tokens

If conversation history grows too long, COMPRESS it: summarize old messages. Like throwing away old receipts but keeping a summary
of your spending.

5. CACHING STRATEGIES (3 levels):
  Level 1 -- EXACT MATCH (Redis): identical query -> cached answer. Fast, simple, limited.
  Level 2 -- SEMANTIC CACHE (embeddings): similar meaning -> cached answer. Smarter, higher hit rate.
  Level 3 -- KV-CACHE (inference engine): vLLM caches computed attention. Automatic, no code needed.
```

🏗 ARCHITECT TIPS + WAR STORIES

These 5 rules are like building codes for LLM applications. You CAN ignore them -- until your app falls over in production.

1. ASYNC is non-negotiable. Synchronous LLM calls = your server handles 5 users instead of 500.
2. CIRCUIT BREAKER: implement with tenacity library (Python). 5 failures in 60 seconds = open circuit.
3. TOKEN BUDGET: plan your token allocation BEFORE writing code. Draw a pie chart.
4. temperature=0 + structured output = most deterministic. Still not 100% -- always handle variation.
5. Three caching levels are complementary. Use all three for maximum savings.

WAR STORY: Startup launched with synchronous LLM calls. Works fine with 10 users. At 100 users, server threads exhausted. Response time: 30+ seconds. Switched to async: same server, 500 concurrent users, 2-second response time. The fix was 20 lines of code (async/await).

📋 POCKET NOTES

5 Design Rules:
  Non-determinism: temp=0 + structured output. Still handle variation.
  Async: LLM calls are I/O-bound. async = 10-50x more concurrent users.
  Circuit breaker: 5 failures -> stop trying -> fallback -> test recovery.
  Token budget: plan allocation. System + context + history + response = total.
  Caching: exact match (Redis) + semantic (embeddings) + KV-cache (engine).
Async: non-negotiable. Sync = 5 users. Async = 500 users.
Idempotency: retry should not create duplicates.
Circuit breaker: CLOSED (normal) -> OPEN (dead) -> HALF-OPEN (testing).
Token budget: compress history when it grows too long.

🎯 INTERVIEW PREP

SAY THIS: 'I design LLM systems async-first because calls are I/O-bound, with circuit breakers for provider resilience, explicit token budgets per request, and tiered caching. Non-determinism is managed through structured outputs and idempotency keys.'

Q: LLM calls are slow. How do you handle scale?
A: 'Async-first: every LLM call is awaited, not blocking. The server handles 500 concurrent users instead of 5. Plus streaming for perceived speed, semantic caching for repeated queries, and model routing so simple queries use fast models.'

Q: Provider is down. Your circuit breaker triggers. Then what?
A: 'Immediately return responses from fallback provider. No queue, no waiting. After 60 seconds, send ONE probe request to the primary. If it succeeds, reset the circuit. Log everything for post-incident review.'

⌨ CODE

Python

```python
import asyncio, time
from dataclasses import dataclass

# CIRCUIT BREAKER
@dataclass
class CircuitBreaker:
    failures: int = 0
    threshold: int = 5
    state: str = "closed"  # closed, open, half-open
    last_failure: float = 0
    recovery_timeout: int = 60  # seconds

    def call(self, func, fallback):
        if self.state == "open":
            if time.time() - self.last_failure > self.recovery_timeout:
                self.state = "half-open"
                print("  Circuit: HALF-OPEN (testing...)")
            else:
                print("  Circuit: OPEN -> immediate fallback")
                return fallback()

        try:
            result = func()
            if self.state == "half-open":
```

```
                self.state = "closed"
                self.failures = 0
                print("  Circuit: recovered -> CLOSED")
            return result
        except Exception:
            self.failures += 1
            self.last_failure = time.time()
            if self.failures >= self.threshold:
                self.state = "open"
                print(f"  Circuit: OPEN after {self.failures} failures")
            return fallback()

# TOKEN BUDGET
def plan_token_budget(max_context=8192):
    budget = {
        "system_prompt": 500,
        "retrieved_docs": 3000,
        "conversation_history": 2000,
        "user_message": 200,
        "response_reserved": 2000,
    }
    total = sum(budget.values())
    remaining = max_context - total
    print("TOKEN BUDGET (8K context):")
    for name, tokens in budget.items():
        pct = tokens/max_context*100
        bar = '#' * int(pct/2)
        print(f"  {name:25s} {tokens:5d} tok ({pct:4.1f}%) {bar}")
    print(f"  {'REMAINING':25s} {remaining:5d} tok ({remaining/max_context*100:.1f}%)")

plan_token_budget()

# ASYNC example
print("\
ASYNC BENEFIT:")
print("  Sync:  5 concurrent users  (thread per request)")
print("  Async: 500 concurrent users (event loop)")
print("  Same server. Same code. 100x more users.")
```

📺 **OUTPUT**

```
TOKEN BUDGET (8K context):
  system_prompt               500 tok ( 6.1%) ###
  retrieved_docs             3000 tok (36.6%) ##################
  conversation_history       2000 tok (24.4%) ############
  user_message                200 tok ( 2.4%) #
  response_reserved          2000 tok (24.4%) ############
  REMAINING                   492 tok ( 6.0%)

ASYNC BENEFIT:
  Sync:  5 concurrent users  (thread per request)
  Async: 500 concurrent users (event loop)
  Same server. Same code. 100x more users.
```

Data Architecture

Embedding pipelines, vector DB schema, hybrid storage, freshness

Imagine building a library from scratch. You need to decide: How do you organize the books when they arrive (embedding pipeline)? How do you arrange the shelves (vector DB schema)? Do you need BOTH a card catalog AND shelf organization (hybrid storage)? And when new editions come out, how do you replace old books (data freshness)? This is data architecture for GenAI.

1. Embedding Pipeline Design | 2. Vector DB Schema + Hybrid Storage + Data Freshness

6. Embedding Pipeline Design

📖 DEEP TEACH

```
TWO PIPELINE MODES:

1. BATCH PIPELINE (the overnight delivery truck):
Imagine a bookstore that receives a delivery truck of 10,000 new books every night. Staff processes them overnight: scan, catalog, shelve. Ready by morning.

Batch embedding: run nightly. Process all new/changed documents at once.
  Source (DB, S3, Drive) -> Extract text -> Chunk -> Embed -> Store in vector DB

Best for: knowledge bases that update daily or weekly. Internal docs, product catalogs.
Cost: cheapest per document (bulk pricing).
Freshness: up to 24 hours stale.

2. REAL-TIME PIPELINE (the instant messenger):
Imagine a newsroom where breaking news must be searchable within seconds of publishing.

Real-time embedding: process each document immediately when created/changed.
  Document created -> webhook/event -> chunk -> embed -> store -> immediately searchable

Best for: customer support (new tickets), news, chat history, real-time data.
Cost: higher per document (no bulk discount, more infrastructure).
Freshness: seconds.

HYBRID (the practical choice):
Batch for the initial load + real-time for incremental updates.
Like the bookstore: overnight truck for big shipments, bike messenger for urgent deliveries.

PIPELINE COMPONENTS:
1. EXTRACTOR: pull text from PDFs, DOCX, HTML, databases. Use Unstructured library.
2. CHUNKER: split into 512-token pieces with 50-token overlap.
3. EMBEDDER: OpenAI text-embedding-3-small or local model.
4. LOADER: write to vector DB (Qdrant, Pinecone, pgvector).
5. METADATA: attach source, date, author, document type. Essential for filtering.

DEDUPLICATION:
Imagine the same document uploaded 3 times. Without dedup, you embed it 3 times (waste), store it 3 times (waste), and retrieve it 3 times (confuses the LLM with 3 identical chunks).
Hash each chunk content. Skip if hash already exists.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of the embedding pipeline like a mail processing center. Mail arrives (documents). It gets sorted (chunked), labeled (embedded), and filed (stored). The question is: do you process once a day (batch) or instantly (real-time)?

1. START with batch. It is simpler, cheaper, and good enough for 80% of use cases.
2. Add real-time only for content that MUST be searchable immediately.
3. METADATA is critical. Always attach: source_url, date_created, document_type, chunk_index. You will need these for filtering and debugging.
4. DEDUPLICATION: hash each chunk. Skip duplicates. Saves 10-30% of embedding costs.
5. IDEMPOTENT: running the pipeline twice should produce the same result, not duplicate everything.
6. MONITORING: track documents processed, errors, embedding latency, storage growth.
```

WAR STORY: Team ran their embedding pipeline daily but had no deduplication. After 6 months, 40% of their vector DB was duplicates. Retrieval quality dropped because the LLM got 3 identical chunks instead of 3 diverse ones. Added content hashing: removed 40% of vectors, retrieval precision improved 15%.

📋 POCKET NOTES

Batch: nightly processing. Cheapest. Up to 24h stale. 80% of use cases.
Real-time: process on creation. Seconds freshness. More infrastructure.
Hybrid: batch for bulk + real-time for urgent. PRACTICAL CHOICE.
Pipeline: extract -> chunk -> embed -> store + metadata.
Metadata: ALWAYS attach source, date, type, chunk_index.
Dedup: hash chunks. Skip duplicates. Saves 10-30%.
Idempotent: run twice = same result (no duplicates).
Monitor: docs processed, errors, latency, storage growth.
Start batch. Add real-time only when freshness matters.

🎯 INTERVIEW PREP

SAY THIS: 'I design embedding pipelines as batch-first with real-time supplements. Batch handles the bulk economically. Real-time webhooks handle urgent updates. Every chunk gets content-hashed for deduplication and tagged with metadata for filtering.'

Q: How do you handle 1 million documents?
A: 'Batch pipeline with parallelism. Split into 100 batches of 10K. Process 10 batches concurrently. Monitor: documents/minute, errors, embedding cost. Total: about 4-6 hours and $100 in embedding cost.'

Q: Document updated. How quickly is it searchable?
A: 'Depends on the mode. Batch: next nightly run (up to 24h). Real-time: webhook triggers immediate re-embedding, searchable in seconds. I choose based on business requirements.'

⌨ CODE

Python

```python
import hashlib, time
from dataclasses import dataclass
from typing import List

@dataclass
class Chunk:
    text: str
    metadata: dict
    hash: str

class EmbeddingPipeline:
    def __init__(self):
        self.processed_hashes = set()  # Deduplication
        self.stats = {"processed":0,"skipped":0,"errors":0}

    def chunk(self, text, chunk_size=500, overlap=50) -> List[str]:
        words = text.split()
        chunks = []
        for i in range(0, len(words), chunk_size - overlap):
            chunks.append(" ".join(words[i:i+chunk_size]))
        return chunks

    def process(self, doc_text, metadata):
        chunks = self.chunk(doc_text)
        results = []
        for i, chunk_text in enumerate(chunks):
            # Deduplication via content hash
            h = hashlib.md5(chunk_text.encode()).hexdigest()
            if h in self.processed_hashes:
                self.stats["skipped"] += 1
                continue
            self.processed_hashes.add(h)

            chunk = Chunk(
                text=chunk_text,
```

```
                metadata={**metadata, "chunk_index":i, "hash":h},
                hash=h
            )
            results.append(chunk)
            self.stats["processed"] += 1
        return results

pipeline = EmbeddingPipeline()

# Process documents
docs = [
    ("Refund policy allows 30 days for returns. " * 50, {"source":"policy.pdf","type":"policy"}),
    ("Pro plan costs $49 per month. Enterprise $199. " * 50, {"source":"pricing.pdf","type":"pricing"}),
    ("Refund policy allows 30 days for returns. " * 50, {"source":"policy_v2.pdf","type":"policy"}),  # Duplicate!
]

for text, meta in docs:
    chunks = pipeline.process(text, meta)
    print(f"  {meta['source']}: {len(chunks)} chunks")

print(f"\
Stats: {pipeline.stats}")
print(f"Dedup saved {pipeline.stats['skipped']} redundant embeddings")
```

📺 **OUTPUT**

```
  policy.pdf: 2 chunks
  pricing.pdf: 2 chunks
  policy_v2.pdf: 0 chunks

Stats: {'processed': 4, 'skipped': 2, 'errors': 0}
Dedup saved 2 redundant embeddings

# policy_v2.pdf was identical content -> 0 chunks processed. Money saved.
```

7. Vector DB Schema + Hybrid Storage + Data Freshness

📖 **DEEP TEACH**

```
3 DATA ARCHITECTURE DECISIONS:

1. VECTOR DB SCHEMA DESIGN:
Imagine organizing a library. You do not just throw books on random shelves. You organize by section (fiction, science), add labels
(author, year), and create an index.

Vector DB schema:
  Collection: 'support_docs' (like a database table)
  Each record:
    vector: [0.12, -0.34, ...] (the embedding)
    text: 'Refund policy is 30 days...' (original content)
    metadata:
      source: 'policy.pdf'
      doc_type: 'policy'
      department: 'support'
      date_updated: '2024-12-01'
      chunk_index: 3

Metadata filtering: 'Find chunks about refunds FROM the policy department UPDATED in the last 30 days.'
This pre-filters BEFORE vector search, making results much more relevant.

MULTI-TENANT: separate collections per customer. Customer A cannot search Customer B's data.

2. HYBRID STORAGE (relational + vector + document):
Imagine a business that needs THREE types of storage:
  PostgreSQL (relational): customer accounts, orders, billing. Structured, transactional.
  Qdrant (vector): document embeddings for RAG search. Semantic search.
  S3/MongoDB (document): original PDF files, images, raw documents.

Each stores what it is best at. They reference each other by ID.
```

Example: user asks about their order. PostgreSQL finds order details. Vector DB finds relevant policy docs. S3 serves the original invoice PDF.

pgvector: PostgreSQL extension that adds vector search. One database for BOTH relational AND vector. Simpler architecture but slower than dedicated vector DBs at scale (10M+ vectors).

3. DATA FRESHNESS AND RE-INDEXING:
Imagine your library has a policy book from 2022. The policy changed in 2024. If you do not replace the old book, customers get wrong answers.

Strategies:
  INCREMENTAL: only re-embed changed documents. Track changes via last_modified timestamp. Like replacing only the updated pages.
  FULL RE-INDEX: re-embed everything. When switching embedding models or major restructuring. Like reorganizing the entire library.
  VERSION TAGGING: keep old and new versions. Switch atomically. Like building a new shelf while the old one is still serving customers, then swapping overnight.

RE-INDEX TRIGGERS:
  - Embedding model change (MUST re-embed everything)
  - Chunking strategy change (affects all chunks)
  - Source document updates (incremental)
  - Scheduled refresh (weekly/monthly full re-index for safety)

🏗 ARCHITECT TIPS + WAR STORIES

Think of data architecture like city planning. You need roads (relational DB for structured data), parks (vector DB for semantic search), and warehouses (document store for raw files). Each serves a purpose.

1. METADATA is your secret weapon. Always store: source, date, type, department. Pre-filtering on metadata before vector search dramatically improves results.
2. pgvector under 10M vectors: use it. One fewer database to manage. Above 10M: dedicated vector DB (Qdrant).
3. MULTI-TENANT: separate collections per customer. Never mix customer data in one collection.
4. VERSION your re-index: build new index alongside old. Swap atomically. Zero downtime.
5. Monitor: vector count growth, query latency by collection size, stale document count.

WAR STORY: RAG system answered 'refund policy is 14 days' when it had changed to 30 days 3 months ago. The old document was still in the vector DB. No freshness mechanism. Added: change detection via last_modified, weekly incremental re-index, and a 'freshness score' that penalizes old documents in ranking. Stale answer rate dropped from 12% to 0.5%.

📋 POCKET NOTES

Vector DB Schema:
  Collection per domain (support_docs, product_docs)
  Each record: vector + text + metadata (source, date, type, department)
  Metadata filtering BEFORE vector search = much better results
  Multi-tenant: separate collections per customer
Hybrid Storage:
  PostgreSQL: structured data (accounts, orders)
  Vector DB: embeddings for RAG (Qdrant, pgvector)
  S3/MongoDB: raw documents (PDFs, images)
  pgvector: one DB for both. Good under 10M vectors.
Data Freshness:
  Incremental: re-embed only changed docs (track last_modified)
  Full re-index: when changing embedding model or chunking
  Version tagging: build new index, swap atomically. Zero downtime.
Monitor: vector count, query latency, stale document count.

🎯 INTERVIEW PREP

SAY THIS: 'I design hybrid storage: PostgreSQL for structured data, dedicated vector DB for semantic search, and document store for raw files. Metadata filtering is critical -- I pre-filter on source, date, and type before vector search. Data freshness via incremental re-indexing with version-tagged atomic swaps.'

Q: How do you handle re-indexing when changing embedding models?
A: 'Full re-index into a new collection. Build alongside the old one. When complete, atomic swap: update the alias to point to the new collection. Zero downtime. The old collection stays as rollback for 48 hours.'

Q: pgvector vs dedicated vector DB?

```
A: 'pgvector under 10M vectors: simpler architecture, one database. Above 10M: dedicated Qdrant or Pinecone for better query performance and specialized features like multi-vector search.'
```

CODE

Python

```python
# Hybrid storage architecture
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from typing import Optional

@dataclass
class VectorDBSchema:
    """Schema for a vector DB collection"""
    collection: str
    vector_dim: int = 1536
    metadata_fields: list = field(default_factory=lambda: [
        "source", "doc_type", "department",
        "date_updated", "chunk_index", "content_hash"
    ])

@dataclass
class HybridArchitecture:
    """Three-store architecture"""
    relational: str = "PostgreSQL"    # Accounts, orders, billing
    vector: str = "Qdrant"            # Embeddings for RAG
    document: str = "S3"              # Raw PDFs, images

    def query_flow(self, user_id, question):
        print(f"  1. [{self.relational}] Get user context: account, plan, history")
        print(f"  2. [{self.vector}] Search relevant docs (filtered by department)")
        print(f"  3. [{self.document}] Fetch original PDF if needed")
        print(f"  4. [LLM] Generate answer from all sources")

# Freshness checker
def check_freshness(docs, max_age_days=30):
    stale = []
    cutoff = datetime.now() - timedelta(days=max_age_days)
    for doc in docs:
        if doc["updated"] < cutoff:
            stale.append(doc)
    return stale

# Demo
print("VECTOR DB SCHEMA:")
schema = VectorDBSchema(collection="support_docs")
print(f"  Collection: {schema.collection}")
print(f"  Dimensions: {schema.vector_dim}")
print(f"  Metadata: {schema.metadata_fields}")

print("\
HYBRID QUERY FLOW:")
arch = HybridArchitecture()
arch.query_flow("user-123", "What is my refund status?")

print("\
FRESHNESS CHECK:")
docs = [{"name":"policy.pdf","updated":datetime(2024,1,1)},
        {"name":"pricing.pdf","updated":datetime.now()}]
stale = check_freshness(docs)
for d in stale: print(f"  STALE: {d['name']} (updated {d['updated'].date()})")
print(f"  {len(stale)}/{len(docs)} documents need re-indexing")
```

OUTPUT

```
VECTOR DB SCHEMA:
  Collection: support_docs
  Dimensions: 1536
  Metadata: ['source', 'doc_type', 'department', 'date_updated', 'chunk_index', 'content_hash']
```

```
HYBRID QUERY FLOW:
  1. [PostgreSQL] Get user context: account, plan, history
  2. [Qdrant] Search relevant docs (filtered by department)
  3. [S3] Fetch original PDF if needed
  4. [LLM] Generate answer from all sources

FRESHNESS CHECK:
  STALE: policy.pdf (updated 2024-01-01)
  1/2 documents need re-indexing

# Three stores, each doing what it does best.
# Metadata filtering + freshness checks = accurate, current answers.
```

LIBRARIES

litellm	Gateway proxy, 100+ providers	`pip install litellm`
gptcache	Semantic caching	`pip install gptcache`
tenacity	Retry + circuit breaker	`pip install tenacity`
qdrant-client	Vector DB (production)	`pip install qdrant-client`
pgvector	PostgreSQL vector extension	`CREATE EXTENSION vector`
unstructured	Document extraction	`pip install unstructured`

ARCHITECTURE DIAGRAM

Renders on GitHub — or paste into mermaid.live

```mermaid
graph TB
    User([User Query]) --> Classifier[BERT Classifier<br/>$0.0001/call - 20ms]

    Classifier -->|Simple 70%| SimplePath
    Classifier -->|Complex 30%| ComplexPath

    subgraph SimplePath[Simple Path - 70% of traffic]
        BM25[BM25 Retriever<br/>FREE - 5ms] --> MiniLLM[GPT-4o-mini<br/>$0.001/call - 800ms]
    end

    subgraph ComplexPath[Complex Path - 30% of traffic]
        HybridSearch[Hybrid Search<br/>$0.0005 - 50ms] --> Rerank[Reranker<br/>$0.001 - 200ms]
        Rerank --> FullLLM[GPT-4o<br/>$0.03/call - 2s]
    end

    MiniLLM --> SafetyCheck
    FullLLM --> SafetyCheck[Safety Scanner<br/>50ms]

    SafetyCheck -->|Pass| SemanticCache[Semantic Cache<br/>Store for next time]
    SafetyCheck -->|Blocked| Fallback[Safe Fallback Response]

    SemanticCache --> Monitor[Cost + Latency Monitor]
    Fallback --> Monitor
    Monitor --> Response([Response to User])

    subgraph Cost Comparison
        direction LR
        Compound[Compound: avg $0.003/query]
        Monolith[Monolith: $0.03/query]
        Savings[Savings: 90%]
    end

    style Classifier fill:#8957e5,color:#fff
    style MiniLLM fill:#1f6feb,color:#fff
    style FullLLM fill:#da3633,color:#fff
    style SafetyCheck fill:#da3633,color:#fff
    style SemanticCache fill:#d29922,color:#fff
    style Monitor fill:#3fb950,color:#fff
    style Compound fill:#3fb950,color:#fff
    style Savings fill:#3fb950,color:#fff,font-weight:bold
```

— POCKET NOTES —

1. Gateway Pattern

```
Gateway: single entry point for all LLM providers.
Analogy: hotel concierge handles all external calls for you.
Handles: unified API, load balancing, fallback, caching, billing.
LiteLLM: standard gateway. 100+ providers. One interface.
  Library mode: import litellm
  Proxy mode: deploy as server, all teams call it.
Benefits: centralized billing, usage quotas, audit logging.
Switch provider = change one config string.
Every production app should have a gateway.
pip install litellm
```

2. Semantic Cache

```
Exact Match: same string -> same answer. Simple. Low hit rate.
Semantic Cache: same MEANING -> same answer. Embedding similarity.
  'refund policy?' matches 'how do returns work?' (same meaning)
How: question -> embed -> search cache (cosine > 0.95) -> hit or miss
Threshold: 0.95 safe | 0.92 risky | below 0.90 too many false matches
Invalidation: time-based (24h) + event-based (data changes)
Cost: 40% semantic hit rate saves ~40% of LLM spend
GPTCache: open-source semantic cache. pip install gptcache.
Start: exact match (Redis). Add semantic if hit rate < 30%.
Separate caches per use case.
```

3. Routing Pattern

```
Routing: classify input -> route to right model/chain.
Analogy: hospital triage nurse sending patients to right department.
3 strategies:
  Rule-based: keywords/length. Zero cost. Predictable.
  Classifier: small ML model. +50ms, +$0.0005. More accurate.
  Cascading: cheap first -> confident? ship it. Unsure? escalate.
Cost: 70% routed to cheap = ~68% savings.
Domain override: medical/legal = ALWAYS best model.
A/B test routing to verify quality maintained.
Track routing decisions + outcomes for improvement.
```

4. Fallback Chain + MapReduce + Iterative Refinement

```
Fallback Chain: GPT-4 -> Claude -> local Llama -> graceful error.
  Automatic switching. User never notices. Test monthly.
  Timeout: 2-3 seconds before switching to fallback.
MapReduce: split big doc -> summarize each piece (parallel) -> combine.
  MAP: cheap model (extract). REDUCE: smart model (synthesize).
  Works for: long docs, codebases, batch analysis.
Iterative Refinement: generate -> critique -> regenerate. Max 3 rounds.
  Best for: reports, code, complex analysis.
  Critic: different model, rules, or same model with critic prompt.
Combine: MapReduce + Refinement = highest quality on long docs.
```

5. Non-Determinism + Async + Circuit Breaker + Token Budgets

```
5 Design Rules:
  Non-determinism: temp=0 + structured output. Still handle variation.
  Async: LLM calls are I/O-bound. async = 10-50x more concurrent users.
  Circuit breaker: 5 failures -> stop trying -> fallback -> test recovery.
  Token budget: plan allocation. System + context + history + response = total.
  Caching: exact match (Redis) + semantic (embeddings) + KV-cache (engine).
Async: non-negotiable. Sync = 5 users. Async = 500 users.
Idempotency: retry should not create duplicates.
Circuit breaker: CLOSED (normal) -> OPEN (dead) -> HALF-OPEN (testing).
Token budget: compress history when it grows too long.
```

6. Embedding Pipeline Design

```
Batch: nightly processing. Cheapest. Up to 24h stale. 80% of use cases.
Real-time: process on creation. Seconds freshness. More infrastructure.
Hybrid: batch for bulk + real-time for urgent. PRACTICAL CHOICE.
Pipeline: extract -> chunk -> embed -> store + metadata.
Metadata: ALWAYS attach source, date, type, chunk_index.
Dedup: hash chunks. Skip duplicates. Saves 10-30%.
Idempotent: run twice = same result (no duplicates).
Monitor: docs processed, errors, latency, storage growth.
Start batch. Add real-time only when freshness matters.
```

7. Vector DB Schema + Hybrid Storage + Data Freshness

```
Vector DB Schema:
  Collection per domain (support_docs, product_docs)
  Each record: vector + text + metadata (source, date, type, department)
  Metadata filtering BEFORE vector search = much better results
  Multi-tenant: separate collections per customer
Hybrid Storage:
  PostgreSQL: structured data (accounts, orders)
  Vector DB: embeddings for RAG (Qdrant, pgvector)
  S3/MongoDB: raw documents (PDFs, images)
  pgvector: one DB for both. Good under 10M vectors.
Data Freshness:
  Incremental: re-embed only changed docs (track last_modified)
  Full re-index: when changing embedding model or chunking
  Version tagging: build new index, swap atomically. Zero downtime.
Monitor: vector count, query latency, stale document count.
```

— INTERVIEW PREP —

1. Gateway Pattern

SAY THIS: 'I implement a Gateway pattern using LiteLLM as a centralized proxy. All teams call the gateway, which handles provider routing, fallback, rate limiting, and cost tracking. Switching providers is a config change, not a code change.'

Q: Why not just call OpenAI directly?
A: 'Three risks: vendor lock-in (switching requires rewriting code), no centralized cost control (runaway spend), and no fallback when OpenAI goes down. A gateway solves all three for minimal overhead.'

Q: How do you manage costs across teams?
A: 'Per-team API keys through the gateway with monthly quotas. Real-time dashboard showing spend by team, model, and endpoint. Alerts at 80% of budget.'

2. Semantic Cache

SAY THIS: 'I implement tiered caching: exact match in Redis for identical queries, semantic cache with embedding similarity for paraphrased queries. Typical hit rates of 30-45% save significant cost and improve latency from 2 seconds to 50ms for cached responses.'

Q: How do you prevent stale cache responses?
A: 'Hybrid invalidation: time-based TTL plus event-driven invalidation when source documents change. For RAG systems, I version-tag cache entries with the document hash -- when docs update, related cache entries auto-expire.'

Q: Similarity threshold -- how do you choose?
A: 'Start at 0.95 (very safe). Measure false positive rate on 1000 queries. If zero false positives, gradually lower to 0.93. Below 0.92, I start seeing different questions matched incorrectly.'

3. Routing Pattern

SAY THIS: 'I implement cost-optimized model routing. Simple queries to GPT-3.5, complex to GPT-4, long-context to Claude. Rule-based routing for quick wins, classifier-based for precision. Domain-critical queries always go to the best model regardless of complexity.'

Q: How do you build the classifier?
A: 'Start with keyword rules: query length, reasoning keywords. Graduate to a fine-tuned classifier trained on 1000 labeled production queries. The classifier itself runs on GPT-3.5 for pennies.'

Q: What if cheap model gives a wrong answer?
A: 'Cascading catches this. Cheap model answers first. I check confidence signals (hedging words, low logprobs). Low confidence triggers automatic escalation to GPT-4. User sees one answer.'

4. Fallback Chain + MapReduce + Iterative Refinement

SAY THIS: 'I design with three resilience patterns: fallback chains for provider outages, MapReduce for documents exceeding context limits, and iterative refinement for quality-critical outputs. Each addresses a different failure mode.'

Q: OpenAI goes down. What happens?
A: 'Automatic failover to Anthropic within 2 seconds. If both down, local Llama. Users experience slightly different response characteristics but zero downtime. I test the chain monthly.'

Q: 500-page document. How to process?
A: 'MapReduce: split into 50-page chunks, summarize each in parallel with GPT-3.5, combine summaries with GPT-4. Parallel MAP means 10 chunks take the same time as 1. Then iterative refinement on the final summary.'

5. Non-Determinism + Async + Circuit Breaker + Token Budgets

SAY THIS: 'I design LLM systems async-first because calls are I/O-bound, with circuit breakers for provider resilience, explicit token budgets per request, and tiered caching. Non-determinism is managed through structured outputs and idempotency keys.'

Q: LLM calls are slow. How do you handle scale?
A: 'Async-first: every LLM call is awaited, not blocking. The server handles 500 concurrent users instead of 5. Plus streaming for perceived speed, semantic caching for repeated queries, and model routing so simple queries use fast models.'

Q: Provider is down. Your circuit breaker triggers. Then what?
A: 'Immediately return responses from fallback provider. No queue, no waiting. After 60 seconds, send ONE probe request to the primary. If it succeeds, reset the circuit. Log everything for post-incident review.'

6. Embedding Pipeline Design

SAY THIS: 'I design embedding pipelines as batch-first with real-time supplements. Batch handles the bulk economically. Real-time webhooks handle urgent updates. Every chunk gets content-hashed for deduplication and tagged with metadata for filtering.'

Q: How do you handle 1 million documents?
A: 'Batch pipeline with parallelism. Split into 100 batches of 10K. Process 10 batches concurrently. Monitor: documents/minute, errors, embedding cost. Total: about 4-6 hours and $100 in embedding cost.'

Q: Document updated. How quickly is it searchable?
A: 'Depends on the mode. Batch: next nightly run (up to 24h). Real-time: webhook triggers immediate re-embedding, searchable in seconds. I choose based on business requirements.'

7. Vector DB Schema + Hybrid Storage + Data Freshness

SAY THIS: 'I design hybrid storage: PostgreSQL for structured data, dedicated vector DB for semantic search, and document store for raw files. Metadata filtering is critical -- I pre-filter on source, date, and type before vector search. Data freshness via incremental re-indexing with version-tagged atomic swaps.'

Q: How do you handle re-indexing when changing embedding models?
A: 'Full re-index into a new collection. Build alongside the old one. When complete, atomic swap: update the alias to point to the new collection. Zero downtime. The old collection stays as rollback for 48 hours.'

Q: pgvector vs dedicated vector DB?
A: 'pgvector under 10M vectors: simpler architecture, one database. Above 10M: dedicated Qdrant or Pinecone for better query performance and specialized features like multi-vector search.'